

DOCUMENTACIÓN DEL TRABAJO DE FIN DE GRADO

Ciclo Superior de Desarrollo de
Aplicaciones Multiplataforma (DAM)

1. PORTADA

Nombre del proyecto: WorkSpace — Plataforma Colaborativa de Gestión de Diarios, Incidencias y Comunicación

Autor: [NOMBRE COMPLETO DEL ALUMNO]

Tutor/a: [NOMBRE DEL TUTOR]

Centro educativo: [NOMBRE DEL CENTRO]

Ciclo formativo: Técnico Superior en Desarrollo de Aplicaciones Multiplataforma (2º DAM)

Curso académico: 2024–2025

Fecha de entrega: Junio 2025

2. ÍNDICE DEL DOCUMENTO

- Portada
- Índice del documento
- Introducción
 - 3.1 Justificación del proyecto
 - 3.2 Análisis comparativo de aplicaciones similares
 - 3.3 Tendencias del mercado

- 3.4 Beneficios y expectativas del proyecto
- 4. Descripción del proyecto
 - 4.1 Tipo de proyecto
 - 4.2 Características principales
 - 4.3 Usuarios destinatarios
- 5. Objetivos del proyecto
 - 5.1 Objetivo general
 - 5.2 Objetivos específicos
- 6. Alcance del proyecto
 - 6.1 Qué incluye el proyecto
 - 6.2 Límites y restricciones
- 7. Requisitos del proyecto
 - 7.1 Requisitos funcionales
 - 7.2 Requisitos técnicos
 - 7.3 Requisitos legales y normativos
- 8. Planificación del proyecto
 - 8.1 Estructura de tareas (WBS)
 - 8.2 Cronograma (Diagrama de Gantt)
 - 8.3 Recursos necesarios
- 9. Plan de gestión de riesgos
 - 9.1 Identificación y evaluación de riesgos
 - 9.2 Recursos preventivos
 - 9.3 Plan de mitigación
- 10. Diseño
 - 10.1 Prototipado y wireframes
 - 10.2 Especificaciones técnicas
 - 10.3 Diagramas UML
- 11. Instalación y preparación
 - 11.1 Procedimientos para poner en marcha el proyecto
 - 11.2 Control de versiones
 - 11.3 Registro de incidencias
- 12. Documentación de ejecución y plan de calidad
 - 12.1 Procedimientos operativos
 - 12.2 Registro de pruebas
 - 12.3 Indicadores de calidad
 - 12.4 Métodos de verificación
- 13. Distribución
 - 13.1 Tecnología de distribución
 - 13.2 Descripción del proceso de distribución
- 14. Manuales
 - 14.1 Manual de instalación
 - 14.2 Manual de usuario
- 15. Conclusiones
 - 15.1 Informe final

- o 15.2 Resultados obtenidos
- o 15.3 Viabilidad del proyecto
- o 15.4 Mejoras futuras

16. Anexos

- o Anexo A: Diagrama Entidad–Relación completo
- o Anexo B: Descripción detallada de la API REST
- o Anexo C: Casos de prueba

17. Índice de tablas e imágenes

18. Bibliografía y referencias

3. INTRODUCCIÓN

3.1 Justificación del proyecto

El presente proyecto surge de la observación de una necesidad real en entornos organizativos y educativos: la dispersión de herramientas para gestionar la comunicación interna, el seguimiento de incidencias y la publicación de contenidos personales o colaborativos. En muchos contextos, estas funciones se gestionan de forma independiente mediante aplicaciones distintas (correo electrónico, hojas de cálculo, gestores de tareas, aplicaciones de mensajería), lo que provoca ineficiencias, falta de trazabilidad y pérdida de información.

La idea de desarrollar **WorkSpace** nace de la voluntad de unificar en una sola plataforma las funcionalidades más demandadas en equipos de trabajo o entornos formativos: un sistema de diarios personales y colaborativos, un gestor de tickets de soporte, un módulo de mensajería interna, un calendario de eventos y un panel de administración con auditoría completa.

El proyecto representa la culminación del ciclo formativo de Desarrollo de Aplicaciones Multiplataforma, aplicando de forma integrada conocimientos de programación backend, desarrollo frontend, diseño de bases de datos, arquitectura de software, seguridad y despliegue en producción. Se ha optado por tecnologías de uso profesional real (Spring Boot, Angular, Ionic, Firebase, PostgreSQL, Docker) para que el resultado sea una aplicación directamente transferible a un contexto laboral.

3.2 Análisis comparativo de aplicaciones similares

Antes de definir el alcance del proyecto, se realizó un análisis de aplicaciones existentes en el mercado que ofrecen funcionalidades similares a las planteadas:

Aplicación	Funcionalidades	Limitaciones frente a WorkSpace
Jira	Gestión de tickets, flujos de trabajo	No incluye diarios, mensajería ni eventos personales

Aplicación	Funcionalidades	Limitaciones frente a WorkSpace
Slack	Mensajería en tiempo real, canales	No tiene gestión de tickets ni diarios con revisión
Notion	Notas, bases de datos, wiki	Sin sistema de tickets ni auditoría por roles
Trello	Gestión visual de tareas	Sin mensajería, sin diarios, sin control de acceso por roles
Microsoft Teams	Comunicación, integración con Office	Alta complejidad, requiere infraestructura Microsoft
Google Workspace	Documentos, correo, calendario	Sin sistema de tickets ni auditoría de acciones

Conclusión del análisis: Ninguna de las aplicaciones analizadas combina en una única plataforma accesible y ligera las cinco funcionalidades centrales del proyecto: diarios con control de visibilidad y colaboración, tickets de soporte con historial, mensajería entre usuarios, eventos con calendario integrado y auditoría de acciones con control por roles.

WorkSpace cubre este nicho con una arquitectura moderna y abierta.

3.3 Tendencias del mercado

El desarrollo de este proyecto se enmarca dentro de varias tendencias tecnológicas actuales:

- **Arquitecturas desacopladas (API-first):** Las aplicaciones modernas separan backend y frontend, permitiendo que distintos clientes consuman la misma API REST. Esta arquitectura facilita la escalabilidad y el mantenimiento, y es la que se ha adoptado en este proyecto.
- **Aplicaciones Progressive Web App (PWA) y móvil-first:** La combinación de Angular e Ionic permite generar aplicaciones web con experiencia similar a una app nativa, adaptable a dispositivos móviles y de escritorio desde una única base de código.
- **Autenticación federada:** El uso de Firebase Authentication con Google Sign-In representa la tendencia hacia sistemas de identidad externos (SSO), más seguros y con menor fricción para el usuario.
- **Contenedorización y DevOps:** El despliegue mediante Docker y Docker Compose es ya un estándar en la industria, permitiendo reproducir entornos de forma fiable y desplegar aplicaciones de manera ágil.
- **Auditoría y trazabilidad:** La creciente importancia del cumplimiento normativo (RGPD, ISO 27001) ha impulsado la inclusión de sistemas de auditoría en las aplicaciones empresariales, algo que WorkSpace implementa de forma nativa.
- **Bases de datos relacionales en la nube:** El uso de PostgreSQL gestionado (servicio Neon) refleja la tendencia hacia bases de datos serverless y gestionadas, que eliminan la carga operativa del administrador.

3.4 Beneficios y expectativas del proyecto

Los principales beneficios esperados del proyecto son:

- **Centralización de herramientas:** Un único punto de acceso para diarios, tickets, mensajes, eventos y notificaciones, reduciendo la fragmentación y mejorando la productividad.
- **Trazabilidad completa:** El sistema de auditoría registra todas las acciones relevantes, permitiendo conocer quién hizo qué y cuándo, lo que resulta esencial para la resolución de incidencias.
- **Control de privacidad:** Los usuarios pueden gestionar sus propios datos y definir la visibilidad de su contenido (privado, pendiente de revisión, público).
- **Escalabilidad técnica:** La arquitectura elegida permite ampliar funcionalidades sin rediseñar el sistema, adaptándose a crecimientos futuros.
- **Aprendizaje aplicado:** El proyecto permite aplicar en un contexto realista tecnologías del nivel de un desarrollador junior-medio en la industria, superando el nivel mínimo esperado en un TFG de DAM.
- **Demostración de competencias:** La implementación completa (backend + frontend + despliegue Docker + base de datos) demuestra la madurez técnica del desarrollador para enfrentarse a proyectos reales.

4. DESCRIPCIÓN DEL PROYECTO

4.1 Tipo de proyecto

WorkSpace es una **aplicación web full-stack** de tipo plataforma colaborativa, orientada a la gestión de equipos de trabajo o entornos formativos. Se trata de un proyecto de desarrollo original que integra:

- Un **backend** basado en microservicio REST (Spring Boot + Kotlin)
- Un **frontend** web/móvil (Angular + Ionic)
- Una **base de datos relacional** (PostgreSQL)
- Un sistema de **autenticación externa** (Firebase Authentication)
- Un entorno de **despliegue containerizado** (Docker + Docker Compose)

La tipología del proyecto es **software de gestión** con componentes de comunicación, contenido y administración. No es un producto comercial, sino un prototipo funcional de nivel profesional desarrollado como Trabajo de Fin de Grado.

4.2 Características principales

Las características principales de la aplicación son:

Sistema de diarios:

- Creación de entradas de diario personal con contenido en texto enriquecido (Markdown)
- Control de visibilidad: privado, pendiente de revisión por staff, público
- Organización por temas (topics) con posibilidad de colaboración
- Sistema de invitaciones para colaborar en temas compartidos
- Comentarios en diarios públicos
- Panel de revisión para el personal de staff

Sistema de tickets de soporte:

- Creación de tickets con título, descripción, prioridad (alta, media, baja) y estado (abierto, en progreso, resuelto)
- Comentarios dentro de cada ticket para la comunicación entre usuario y staff
- Historial completo de cambios de estado para auditoría
- Panel de administración de tickets con filtros y gestión masiva

Mensajería interna:

- Conversaciones individuales y grupales entre usuarios
- Envío de mensajes con control de lectura (leído/no leído)
- Gestión de participantes: añadir y eliminar
- Bandeja de entrada para el staff (soporte chat)

Gestión de eventos:

- Creación de eventos con fecha, hora, título y descripción
- Visibilidad privada o pública
- Vista de calendario integrada (Syncfusion Schedule + Angular Calendar)

Sistema de notificaciones:

- Notificaciones push mediante Firebase Cloud Messaging (FCM)
- Centro de notificaciones en la aplicación
- Marcado de notificaciones como leídas

Panel de administración:

- Gestión completa de usuarios: ver, editar, eliminar, cambiar rol
- Gestión de departamentos y estructura organizativa
- Revisión y aprobación de diarios públicos
- Supervisión de tickets, eventos y mensajes de soporte
- Registro de auditoría con niveles de severidad (INFO, WARNING, DANGER)

Funcionalidades transversales:

- Autenticación mediante Google (Firebase OAuth 2.0)
- Control de acceso basado en roles (ADMIN, STAFF, USER)
- Perfil de usuario personalizable (nombre, biografía, foto)
- Control de privacidad de contacto

4.3 Usuarios destinatarios

La aplicación está diseñada para tres perfiles de usuario:

Usuario estándar (rol USER): Personas que utilizan la plataforma para gestionar su contenido personal, comunicarse con otros usuarios y solicitar soporte a través del sistema de tickets. Puede ser un empleado, estudiante o miembro de una organización.

Personal de soporte (rol STAFF): Personas responsables de atender las solicitudes de los usuarios, revisar el contenido publicado y gestionar las comunicaciones de soporte. Dispone de permisos adicionales para moderar contenido y actualizar el estado de tickets.

Administrador (rol ADMIN): Persona responsable de la gestión global de la plataforma: usuarios, departamentos, configuración general y supervisión de todas las actividades mediante el panel de auditoría.

5. OBJETIVOS DEL PROYECTO

5.1 Objetivo general

Diseñar, implementar y desplegar una plataforma web colaborativa completa que integre en un único sistema la gestión de diarios personales y colaborativos, tickets de soporte, mensajería interna, eventos y auditoría, aplicando arquitecturas modernas y buenas prácticas de desarrollo de software profesional.

5.2 Objetivos específicos

1. **Diseñar la arquitectura del sistema** separando claramente frontend y backend mediante una API REST, siguiendo el patrón cliente-servidor desacoplado.
2. **Implementar el backend** con Spring Boot y Kotlin, organizando el código en capas (controladores, servicios, repositorios, DTOs) y documentando la API con OpenAPI/Swagger.
3. **Diseñar y normalizar la base de datos relacional** con PostgreSQL, aplicando las formas normales hasta 3FN y garantizando la integridad referencial mediante claves foráneas.
4. **Implementar el frontend** con Angular e Ionic, desarrollando una interfaz responsiva y adaptada a dispositivos móviles y de escritorio.
5. **Integrar un sistema de autenticación seguro** mediante Firebase Authentication con Google Sign-In y tokens JWT para proteger los endpoints del backend.

6. **Implementar control de acceso basado en roles** (ADMIN, STAFF, USER) que limite las funcionalidades disponibles para cada perfil.
 7. **Desarrollar el módulo de diarios** con control de visibilidad, sistema de revisión y colaboración entre usuarios.
 8. **Desarrollar el módulo de tickets** con gestión de estados, prioridades, comentarios e historial de cambios.
 9. **Desarrollar el módulo de mensajería** con soporte para conversaciones individuales y grupales.
 10. **Desarrollar el módulo de eventos** con calendario integrado y control de visibilidad.
 11. **Implementar un sistema de auditoría** que registre las acciones relevantes realizadas en el sistema, incluyendo actor, acción, recurso y severidad.
 12. **Implementar notificaciones push** mediante Firebase Cloud Messaging para alertar a los usuarios de eventos relevantes en tiempo real.
 13. **Desplegar el sistema completo** mediante Docker y Docker Compose, garantizando la reproducibilidad del entorno en producción.
 14. **Realizar pruebas funcionales y de seguridad** que validen el correcto funcionamiento del sistema antes de su puesta en producción.
-

6. ALCANCE DEL PROYECTO

6.1 Qué incluye el proyecto

El alcance del proyecto comprende el desarrollo e implantación de los siguientes elementos:

Backend (API REST):

- 15 controladores REST con sus respectivos endpoints (usuarios, roles, departamentos, diarios, temas de diario, colaboraciones, tickets, comentarios de tickets, historial de tickets, conversaciones, participantes, mensajes, eventos, notificaciones, auditoría)
- 13 servicios de negocio
- 16 entidades JPA mapeadas a tablas de base de datos
- Sistema de autenticación y autorización mediante Firebase JWT
- Documentación automática con SpringDoc OpenAPI (Swagger UI)
- Sistema de auditoría mediante Spring AOP
- Programación de tareas con Spring Scheduling

Base de datos:

- Diseño e implementación del esquema relacional completo con 16 tablas
- Índices en campos de búsqueda frecuente
- Gestión mediante PostgreSQL (servicio Neon en la nube)

Frontend:

- Aplicación Angular/Ionic con enrutamiento protegido por guards
- Páginas públicas: inicio, blog, FAQ, contacto, privacidad
- Sección de usuario: perfil, diario, tickets, mensajes, eventos, notificaciones
- Panel de administración completo: gestión de usuarios, tickets, diarios, eventos, mensajes de soporte, auditoría, departamentos
- Interceptor HTTP para gestión automática de tokens de autenticación
- Integración con Firebase (autenticación, FCM para notificaciones)
- Visualización de calendario con Syncfusion Schedule

Despliegue:

- Dockerfiles para backend y frontend
- Docker Compose para orquestación de todos los servicios
- Build de producción del frontend con Nginx

Documentación:

- Documentación técnica completa del proyecto
- Manual de instalación y uso

6.2 Límites y restricciones

Fuera del alcance:

- Desarrollo de aplicaciones móviles nativas (iOS/Android). El frontend Ionic genera una PWA, no una app distribuida en tiendas de aplicaciones.
- Integración con sistemas ERP, CRM u otros sistemas externos no mencionados.
- Panel de analítica avanzada o business intelligence.
- Sistema de facturación o pagos.
- Soporte para múltiples idiomas (i18n). La aplicación está íntegramente en español.
- Tests automatizados completos (unitarios e integración con cobertura >80%). Se realizan pruebas funcionales manuales.
- Pipeline CI/CD automatizado (GitHub Actions, Jenkins, etc.).

Restricciones técnicas:

- La autenticación requiere cuenta Google (no hay registro con email/contraseña propio).
- El sistema de mensajería no es en tiempo real puro (WebSockets); la actualización se realiza mediante polling o recarga.

- El almacenamiento de archivos adjuntos no está implementado en la versión actual.

Restricciones de recursos:

- Desarrollo realizado por un único desarrollador durante el periodo del TFG.
 - Infraestructura limitada al plan disponible del servicio de base de datos en la nube.
-

7. REQUISITOS DEL PROYECTO

7.1 Requisitos funcionales

Los requisitos funcionales describen el comportamiento del sistema desde el punto de vista del usuario.

Gestión de usuarios

- **RF-01:** El sistema debe permitir que los usuarios se registren mediante su cuenta de Google (Firebase Authentication).
- **RF-02:** El sistema debe asignar automáticamente el rol USER a los nuevos registros, salvo asignación manual por parte de un administrador.
- **RF-03:** El usuario debe poder consultar y modificar su perfil (nombre, biografía, foto de perfil).
- **RF-04:** El usuario debe poder configurar sus preferencias de contacto (permitir/no permitir que otros usuarios le contacten).
- **RF-05:** Los administradores deben poder gestionar usuarios: crear, ver, editar rol/departamento y eliminar.
- **RF-06:** El sistema debe permitir organizar usuarios por departamentos.

Sistema de diarios

- **RF-07:** El usuario debe poder crear entradas de diario asociadas a un tema.
- **RF-08:** El usuario debe poder definir la visibilidad de cada entrada: privada, pendiente de revisión o pública.
- **RF-09:** El sistema debe permitir crear temas de diario e invitar a otros usuarios a colaborar en ellos.
- **RF-10:** Los usuarios invitados deben poder aceptar o rechazar invitaciones de colaboración.
- **RF-11:** El personal de staff debe poder revisar y aprobar o rechazar las entradas marcadas como "pendiente".
- **RF-12:** Los usuarios deben poder comentar en los diarios públicos.

Sistema de tickets

- **RF-13:** El usuario debe poder crear tickets de soporte con título, descripción y prioridad.
- **RF-14:** El sistema debe mostrar al usuario sus propios tickets con su estado actual.

- **RF-15:** El personal autorizado (STAFF/ADMIN) debe poder cambiar el estado de un ticket (abierto → en progreso → resuelto).
- **RF-16:** El sistema debe registrar el historial completo de cambios de estado de cada ticket.
- **RF-17:** Los usuarios deben poder añadir comentarios a sus tickets.

Mensajería

- **RF-18:** El sistema debe permitir crear conversaciones individuales o grupales.
- **RF-19:** Los participantes de una conversación deben poder enviar y recibir mensajes.
- **RF-20:** El sistema debe registrar el estado de lectura de los mensajes.
- **RF-21:** Los administradores deben poder gestionar una bandeja de soporte para atender a los usuarios.

Eventos

- **RF-22:** El usuario debe poder crear eventos con fecha, hora, título, descripción y visibilidad.
- **RF-23:** El sistema debe mostrar los eventos en una vista de calendario.
- **RF-24:** El usuario debe poder eliminar sus propios eventos.

Notificaciones

- **RF-25:** El sistema debe enviar notificaciones push cuando se produzcan eventos relevantes (nuevo mensaje, respuesta a ticket, etc.).
- **RF-26:** El usuario debe poder consultar y marcar como leídas sus notificaciones.

Auditoría

- **RF-27:** El sistema debe registrar automáticamente las acciones relevantes (login, creación, modificación, eliminación de recursos).
- **RF-28:** Los administradores deben poder consultar el registro de auditoría con filtros.

7.2 Requisitos técnicos

Los requisitos técnicos definen las condiciones técnicas que debe cumplir el sistema.

Backend

- **RT-01:** El backend debe estar desarrollado con **Spring Boot 3.x** y **Kotlin**.
- **RT-02:** El backend debe exponer una **API REST** que cumpla con los principios RESTful (recursos identificables, métodos HTTP correctos, respuestas JSON, stateless).
- **RT-03:** La API debe estar documentada automáticamente con **SpringDoc OpenAPI 3.0** (Swagger UI).
- **RT-04:** La autenticación debe basarse en **tokens JWT** generados por Firebase Authentication.
- **RT-05:** El sistema debe implementar autorización por roles mediante **Spring Security** y anotaciones `@PreAuthorize`.

- **RT-06:** La arquitectura del backend debe seguir el patrón de capas: Controller → Service → Repository → Entity/DTO.
- **RT-07:** El sistema de auditoría debe implementarse mediante **Spring AOP** con aspecto transversal.
- **RT-08:** Las notificaciones push deben enviarse mediante **Firebase Cloud Messaging (FCM)**.

Base de datos

- **RT-09:** La base de datos debe ser **PostgreSQL** (versión 14 o superior).
- **RT-10:** El esquema debe diseñarse aplicando normalización hasta la **Tercera Forma Normal (3FN)**.
- **RT-11:** Deben definirse **índices** sobre los campos utilizados frecuentemente en consultas y filtros.
- **RT-12:** La gestión del esquema debe realizarse mediante **Hibernate/JPA** con `ddl-auto: update`.

Frontend

- **RT-13:** El frontend debe estar desarrollado con **Angular 20** e **ionic 8**.
- **RT-14:** La aplicación debe ser **responsiva** y adaptarse a pantallas de móvil, tablet y escritorio.
- **RT-15:** La navegación debe estar protegida mediante **guards de Angular** que comprueben autenticación y rol.
- **RT-16:** Las peticiones HTTP al backend deben incluir automáticamente el token JWT mediante un **interceptor Angular**.
- **RT-17:** La integración de calendario debe realizarse con **Syncfusion EJ2 Schedule**.

Despliegue

- **RT-18:** Todos los componentes (backend, frontend, base de datos) deben poder desplegarse mediante **Docker** y **Docker Compose**.
- **RT-19:** El frontend debe compilarse para producción y servirse mediante **Nginx**.
- **RT-20:** La configuración sensible (credenciales, claves) debe gestionarse mediante **variables de entorno**, sin incluirla en el código fuente.

Rendimiento y seguridad

- **RT-21:** Los listados extensos deben implementar **paginación** para optimizar el rendimiento.
- **RT-22:** Las peticiones protegidas deben devolver **HTTP 401** si no se proporciona token válido y **HTTP 403** si el usuario no tiene permisos suficientes.
- **RT-23:** Todos los datos de entrada deben ser **validados** en el backend antes de procesarse.

7.3 Requisitos legales y normativos

Protección de datos (RGPD)

- **RL-01:** La aplicación debe informar a los usuarios sobre el tratamiento de sus datos personales mediante una política de privacidad accesible desde la interfaz.

- **RL-02:** Los datos personales de los usuarios (nombre, email, foto de perfil) provienen de la cuenta de Google mediante OAuth 2.0, y su tratamiento está sujeto a las condiciones de Google y al RGPD.
- **RL-03:** Los usuarios deben poder eliminar su propia cuenta y datos desde la aplicación (derecho de supresión — RGPD Art. 17). El endpoint `DELETE /api/usuarios/perfil` implementa esta funcionalidad.
- **RL-04:** Los datos no deben compartirse con terceros no autorizados ni usarse con fines distintos a los declarados.
- **RL-05:** La aplicación debe implementar medidas técnicas de seguridad adecuadas (autenticación, HTTPS, control de acceso) para proteger los datos personales.

Licencias de software

- **RL-06:** Todas las dependencias y librerías utilizadas (Spring Boot, Angular, Ionic, Firebase, Syncfusion) deben usarse respetando sus licencias. Syncfusion requiere licencia de uso, que ha sido registrada en el proyecto.
- **RL-07:** El software desarrollado es un proyecto académico sin fines comerciales. Su distribución y uso están sujetos a las condiciones establecidas por el centro educativo.

Accesibilidad

- **RL-08:** La aplicación debe seguir, en la medida de lo posible, las pautas de accesibilidad WCAG 2.1 nivel A, garantizando que la interfaz sea usable por el mayor número posible de personas.

8. PLANIFICACIÓN DEL PROYECTO

8.1 Estructura de tareas (WBS — Work Breakdown Structure)

El proyecto se ha dividido en las siguientes fases y tareas principales:

FASE 1: Análisis y diseño (Semanas 1–3)

- 1.1 Análisis de requisitos funcionales y no funcionales
- 1.2 Análisis comparativo de aplicaciones similares
- 1.3 Diseño del modelo de datos (entidades y relaciones)
- 1.4 Diseño de la arquitectura del sistema
- 1.5 Diseño de los wireframes de la interfaz de usuario
- 1.6 Definición de la API (endpoints, métodos, DTOs)
- 1.7 Planificación del proyecto y gestión de riesgos

FASE 2: Configuración del entorno (Semana 3–4)

- 2.1 Configuración del repositorio de control de versiones
- 2.2 Configuración del proyecto Spring Boot (Kotlin)
- 2.3 Configuración del proyecto Angular/Ionic
- 2.4 Configuración de Firebase (Authentication + FCM)
- 2.5 Creación y configuración de la base de datos PostgreSQL (Neon)
- 2.6 Configuración del entorno de desarrollo local

FASE 3: Desarrollo del backend (Semanas 4–10)

- 3.1 Implementación del modelo de datos (entidades JPA)
- 3.2 Implementación de repositorios (Spring Data JPA)
- 3.3 Implementación de la seguridad (Firebase JWT, Spring Security)
- 3.4 Implementación del módulo de usuarios y roles
- 3.5 Implementación del módulo de diarios y colaboraciones
- 3.6 Implementación del módulo de tickets y comentarios
- 3.7 Implementación del módulo de mensajería
- 3.8 Implementación del módulo de eventos
- 3.9 Implementación del módulo de notificaciones y FCM
- 3.10 Implementación del sistema de auditoría (Spring AOP)
- 3.11 Configuración de SpringDoc OpenAPI (Swagger UI)
- 3.12 Pruebas de endpoints con Swagger

FASE 4: Desarrollo del frontend (Semanas 8–14)

- 4.1 Configuración del routing y guards de Angular
- 4.2 Implementación del interceptor HTTP para tokens JWT
- 4.3 Desarrollo de las páginas públicas (inicio, blog, FAQ, contacto, privacidad)
- 4.4 Desarrollo del flujo de autenticación (Google Sign-In)
- 4.5 Desarrollo de la sección de perfil de usuario
- 4.6 Desarrollo del módulo de diarios (lista, detalle, creación, temas, invitaciones)
- 4.7 Desarrollo del módulo de tickets
- 4.8 Desarrollo del módulo de mensajería
- 4.9 Desarrollo del módulo de eventos y calendario
- 4.10 Desarrollo del centro de notificaciones
- 4.11 Desarrollo del panel de administración completo
- 4.12 Integración con Firebase (autenticación, FCM)
- 4.13 Ajustes de diseño responsivo (móvil/tablet/escritorio)

FASE 5: Integración y pruebas (Semanas 14–16)

- 5.1 Pruebas de integración frontend–backend
- 5.2 Pruebas funcionales de cada módulo

- 5.3 Pruebas de autenticación y autorización por roles
- 5.4 Pruebas de validación de datos
- 5.5 Corrección de errores detectados
- 5.6 Pruebas de usabilidad

FASE 6: Despliegue (Semanas 16–17)

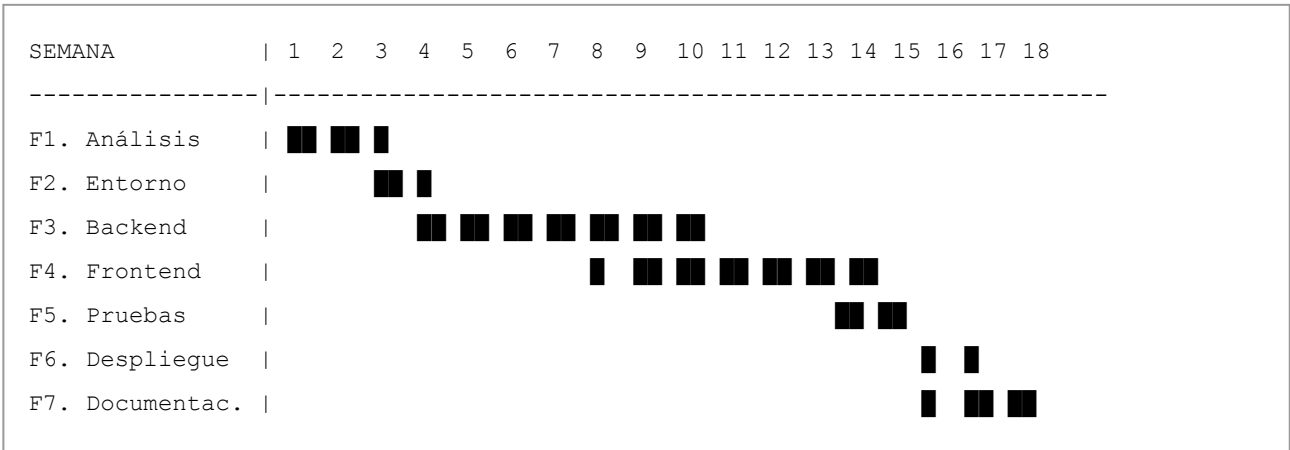
- 6.1 Creación de Dockerfile para el backend
- 6.2 Creación de Dockerfile para el frontend (build + Nginx)
- 6.3 Configuración de Docker Compose
- 6.4 Pruebas del despliegue local con Docker Compose
- 6.5 Despliegue en entorno de producción
- 6.6 Pruebas de validación en producción

FASE 7: Documentación y entrega (Semanas 17–18)

- 7.1 Redacción de la documentación técnica completa
- 7.2 Redacción del manual de instalación y uso
- 7.3 Preparación de la defensa oral
- 7.4 Revisión final y entrega

8.2 Cronograma (Diagrama de Gantt)

El proyecto se planificó para un período de 18 semanas (aproximadamente 4,5 meses), distribuidas de la siguiente manera:



Nota: Las fases de backend y frontend se solapan a partir de la semana 8, siguiendo un enfoque iterativo que permite desarrollar en paralelo ambas capas una vez establecidas las interfaces.

8.3 Recursos necesarios

Recursos técnicos — Hardware

- Ordenador de desarrollo con mínimo 16 GB de RAM (necesario para ejecutar Docker, el servidor Spring Boot, el servidor de desarrollo Angular y el servidor de base de datos simultáneamente)
- Conexión a Internet para acceso a los servicios en la nube (Firebase, Neon PostgreSQL)

Recursos técnicos — Software (todos gratuitos o con licencia de estudiante)

Herramienta	Versión	Uso
IntelliJ IDEA / VS Code	Última	IDE de desarrollo
Java JDK	17 (LTS)	Compilación del backend
Kotlin	1.9.25	Lenguaje del backend
Maven	3.9	Gestión de dependencias backend
Node.js	20 (LTS)	Entorno de ejecución frontend
Angular CLI	20	Scaffolding y build del frontend
Ionic CLI	8	Framework UI
Docker Desktop	Última	Contenedorización
Git + GitHub	Última	Control de versiones
Postman / Swagger UI	Última	Prueba de endpoints
pgAdmin	4	Gestión de base de datos
Firebase Console	Web	Configuración de servicios Firebase

Recursos técnicos — Servicios en la nube

Servicio	Plan	Uso
Firebase Authentication	Gratuito (Spark)	Autenticación de usuarios
Firebase Cloud Messaging	Gratuito	Notificaciones push
Neon PostgreSQL	Gratuito (tier básico)	Base de datos en la nube
GitHub	Gratuito	Repositorio de código

Recursos humanos

- 1 desarrollador (el alumno): responsable de análisis, diseño, desarrollo, pruebas y documentación
- Tutor del centro: orientación y revisión del proyecto

9. PLAN DE GESTIÓN DE RIESGOS

9.1 Identificación y evaluación de riesgos

Se han identificado los principales riesgos que podrían afectar al desarrollo del proyecto, evaluados según su probabilidad e impacto (escala 1–5):

ID	Riesgo	Probabilidad	Impacto	Criticidad
R-01	Cambios en la API de Firebase que rompan la autenticación	2	5	Alta
R-02	Incompatibilidades de versiones entre Angular/Ionic y dependencias	3	4	Alta
R-03	Pérdida de datos en la base de datos de desarrollo	2	4	Media-Alta
R-04	Subestimación del tiempo de desarrollo de algún módulo	4	3	Alta
R-05	Limitaciones del plan gratuito de Neon (tamaño, conexiones)	3	3	Media
R-06	Problemas de CORS entre frontend y backend en producción	3	3	Media
R-07	Errores de seguridad (acceso no autorizado a recursos)	2	5	Alta
R-08	Problemas en la configuración de Docker en producción	3	3	Media
R-09	Cambios de diseño tardíos que requieran refactorización	3	2	Media
R-10	Ausencia de soporte técnico (tutor, documentación) para una tecnología concreta	2	3	Baja-Media

9.2 Recursos preventivos

Para cada riesgo identificado se han definido las siguientes medidas preventivas:

- **R-01 (Firebase API):** Seguir la documentación oficial de Firebase y establecer dependencias con versiones fijas en el `pom.xml` y `package.json`. Monitorizar el canal de cambios de Firebase.
- **R-02 (Incompatibilidades):** Utilizar versiones LTS estables de Node.js y JDK. Actualizar dependencias de forma controlada, probando en entorno local antes de integrar.
- **R-03 (Pérdida de datos):** Realizar backups regulares de la base de datos de desarrollo. Mantener scripts de inicialización para recrear datos de prueba.
- **R-04 (Tiempo):** Planificación con holgura entre fases. Priorizar las funcionalidades core del MVP; las funcionalidades secundarias se implementan si el tiempo lo permite.
- **R-05 (Neon limitaciones):** Monitorizar el uso del almacenamiento. Disponer de un plan de migración a instancia local de PostgreSQL si fuera necesario.
- **R-06 (CORS):** Configurar correctamente la política CORS en Spring Boot desde el inicio, especificando los orígenes permitidos. Probar en entorno de integración antes de producción.

- **R-07 (Seguridad):** Implementar pruebas específicas de autorización. Revisar sistemáticamente los endpoints con herramientas como Swagger antes de desplegar.
- **R-08 (Docker):** Probar el despliegue con Docker Compose en local antes de llevar a producción. Documentar todos los pasos de configuración.
- **R-09 (Diseño):** Definir y validar los wireframes y la arquitectura antes de comenzar el desarrollo. Usar un enfoque iterativo para ajustar el diseño sin grandes refactorizaciones.
- **R-10 (Soporte técnico):** Consultar la documentación oficial de cada tecnología. Usar comunidades como Stack Overflow, GitHub Issues y foros especializados.

9.3 Plan de mitigación

En caso de que un riesgo se materialice, se aplicarán las siguientes acciones:

ID	Riesgo materializado	Plan de mitigación
R-01	Firebase API cambia	Migrar a versión compatible o buscar alternativa de autenticación JWT propia
R-02	Incompatibilidad de versiones	Retroceder a la versión anterior estable de la dependencia afectada; aislar el cambio
R-03	Pérdida de datos	Restaurar desde el último backup disponible; recrear datos de prueba con scripts
R-04	Tiempo insuficiente	Reducir el alcance a las funcionalidades del MVP; posponer las secundarias a una futura versión
R-05	Neon lleno	Migrar la base de datos a una instancia local de PostgreSQL en Docker
R-06	CORS en producción	Ajustar la configuración CORS del backend para el dominio de producción correcto
R-07	Brecha de seguridad	Identificar el endpoint afectado, aplicar corrección y volver a probar
R-08	Docker falla	Revisar logs del contenedor; consultar documentación; desplegar sin Docker si es necesario
R-09	Rediseño tardío	Implementar el cambio de diseño de forma incremental, priorizando la funcionalidad
R-10	Sin soporte técnico	Ampliar búsqueda a documentación oficial, GitHub, Stack Overflow, foros de la comunidad

10. DISEÑO

10.1 Prototipado y wireframes

El diseño de la interfaz de usuario se ha planificado siguiendo un enfoque mobile-first, aprovechando las capacidades de Ionic para generar interfaces adaptadas a dispositivos móviles y de escritorio.

Páginas públicas

Las páginas públicas (inicio, blog, FAQ, contacto, privacidad) siguen una estructura de landing page con barra de navegación superior, secciones de contenido y pie de página. El acceso a la aplicación se realiza mediante el botón de inicio de sesión con Google.

Dashboard de usuario

Una vez autenticado, el usuario accede a su panel personal con las siguientes secciones accesibles desde el menú lateral:

- **Perfil:** Datos personales y configuración de privacidad
- **Diario:** Lista de entradas, temas y colaboraciones
- **Tickets:** Mis solicitudes de soporte
- **Mensajes:** Conversaciones activas
- **Eventos:** Calendario personal
- **Notificaciones:** Centro de alertas

Panel de administración

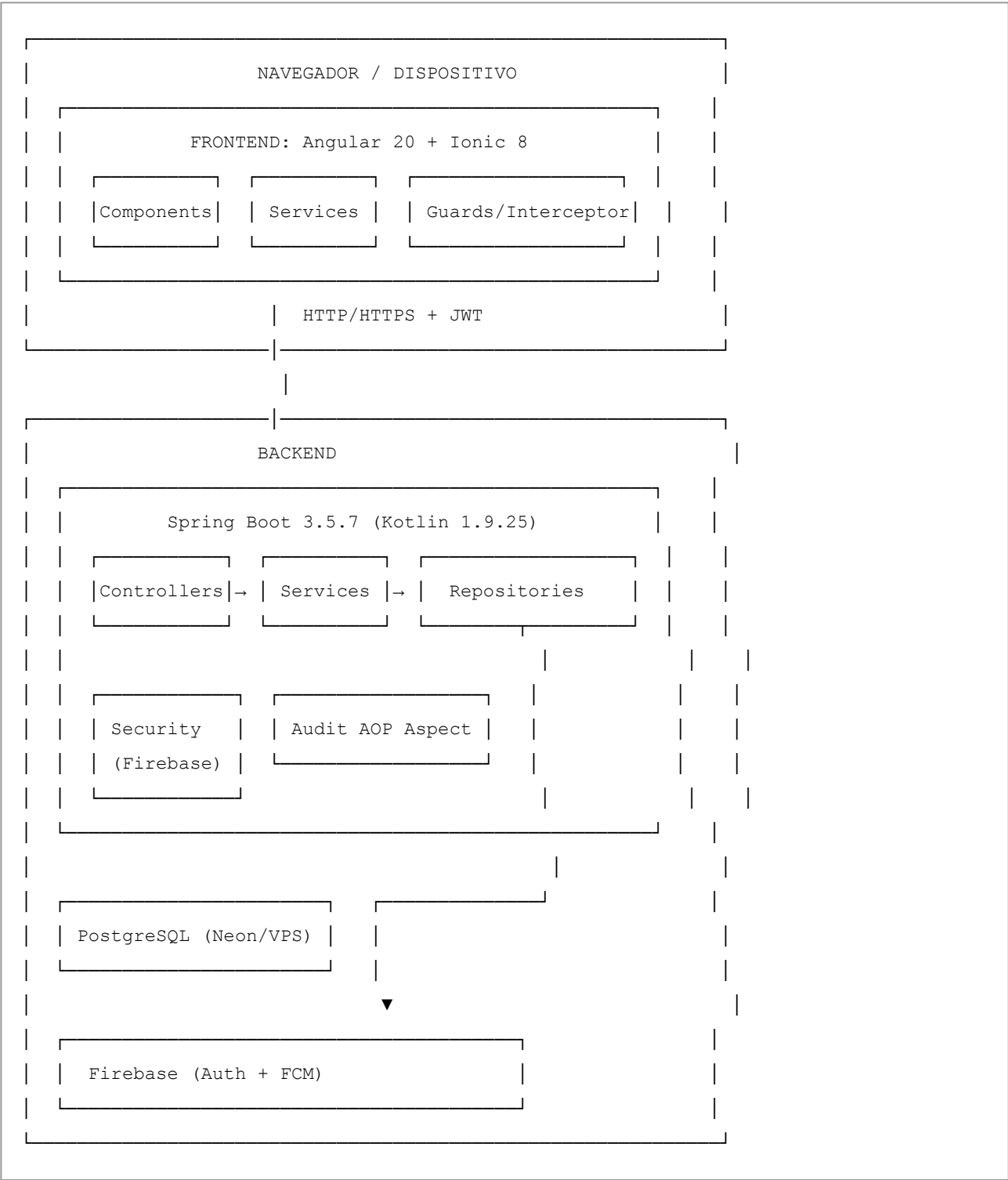
El administrador y el staff acceden a un panel diferenciado con:

- **Usuarios:** Tabla de gestión con filtros y acciones
- **Tickets (admin):** Vista global de todos los tickets con panel de gestión
- **Diarios (admin):** Cola de revisión de diarios pendientes
- **Mensajes de soporte:** Bandeja de entrada de soporte
- **Auditoría:** Registro de acciones del sistema
- **Personal:** Gestión de departamentos y asignación de roles

Los wireframes detallados se encuentran en las capturas de pantalla adjuntas en la carpeta /documentación del proyecto.

10.2 Especificaciones técnicas

Arquitectura global del sistema



Stack tecnológico completo

Capa	Tecnología	Versión
Lenguaje backend	Kotlin	1.9.25
Framework backend	Spring Boot	3.5.7
ORM	Spring Data JPA / Hibernate	Incluido en Spring Boot
Base de datos	PostgreSQL	14+
Autenticación	Firebase Authentication	SDK 9.x
Notificaciones push	Firebase Cloud Messaging	SDK 9.x

Capa	Tecnología	Versión
Documentación API	SpringDoc OpenAPI	2.x
Build backend	Maven	3.9
Lenguaje frontend	TypeScript	5.x
Framework frontend	Angular	20.0.0
UI Framework	Ionic	8.0.0
Calendario	Syncfusion EJ2 Schedule	Última
Contenedorización	Docker + Docker Compose	Última
Servidor web (prod.)	Nginx	Alpine
Runtime frontend	Node.js	20 LTS
JDK	Java	17 LTS

10.3 Diagramas UML

Diagrama de Casos de Uso (resumen)

Actores: Usuario (USER), Staff (STAFF), Administrador (ADMIN)

USUARIO puede:

- Autenticarse con Google
- Ver/editar su perfil
- Crear/editar/eliminar diarios (propios)
- Crear temas de diario e invitar colaboradores
- Crear/ver tickets propios y añadir comentarios
- Enviar/recibir mensajes
- Crear/ver/eliminar eventos propios
- Ver notificaciones

STAFF puede (además de lo anterior):

- Ver todos los diarios públicos y pendientes
- Aprobar/rechazar diarios pendientes
- Ver y comentar todos los tickets
- Cambiar estado de tickets
- Acceder a la bandeja de soporte

ADMINISTRADOR puede (además de lo anterior):

- Gestionar todos los usuarios (CRUD)
- Gestionar departamentos
- Ver todos los tickets y eventos
- Ver registro de auditoría
- Asignar roles a usuarios

Diagrama de Flujo — Autenticación

```
[Usuario] → Pulsa "Iniciar sesión con Google"
  → Firebase abre popup OAuth
  → Usuario concede permisos
  → Firebase devuelve token JWT
  → Frontend envía token a POST /api/auth/google
  → Backend verifica token con Firebase Admin SDK
  → Backend busca usuario por firebase_uid en BD
    → Si existe: devuelve datos del usuario
    → Si no existe: crea nuevo usuario con rol USER
  → Frontend almacena token y redirige al dashboard
```

Diagrama de flujo — Ciclo de vida de un ticket

```
[Usuario] Crea ticket → Estado: ABIERTO
  → [Staff/Admin] Asigna → Estado: EN_PROGRESO
    → Se registra en TicketHistorico
    → Se genera notificación al usuario
  → [Staff/Admin] Resuelve → Estado: RESUELTO
    → Se registra en TicketHistorico
    → Se genera notificación al usuario
```

Modelo Entidad–Relación (simplificado)

Las entidades principales del sistema y sus relaciones son:

- **Usuario** (1) ↔ (N) **Ticket** (un usuario crea múltiples tickets)
- **Ticket** (1) ↔ (N) **TicketComentario**
- **Ticket** (1) ↔ (N) **TicketHistorico**
- **Usuario** (1) ↔ (N) **Diario**
- **DiarioTema** (1) ↔ (N) **Diario**
- **DiarioTema** (1) ↔ (N) **DiarioColaboracion** ↔ (1) **Usuario**
- **Diario** (1) ↔ (N) **DiarioComentario**
- **Usuario** (N) ↔ (N) **Conversacion** (a través de ConversacionParticipante)
- **Conversacion** (1) ↔ (N) **Mensaje**
- **Usuario** (1) ↔ (N) **Evento**
- **Usuario** (1) ↔ (N) **Notificacion**
- **Usuario** (N) ↔ (1) **Rol**
- **Usuario** (N) ↔ (1) **Departamento**
- **Sistema** (1) ↔ (N) **Auditoria**

11. INSTALACIÓN Y PREPARACIÓN

11.1 Procedimientos necesarios para hacer funcionar el proyecto

Prerrequisitos del entorno de desarrollo

Para trabajar con el proyecto en local se necesitan los siguientes componentes instalados:

1. **Java JDK 17** (OpenJDK o Temurin): `java -version` debe mostrar versión 17.x
2. **Maven 3.9+**: `mvn -version`
3. **Node.js 20 LTS**: `node -v` debe mostrar v20.x
4. **Angular CLI 20**: `npm install -g @angular/cli`
5. **Ionic CLI**: `npm install -g @ionic/cli`
6. **Docker Desktop** (para despliegue containerizado)
7. **Git**: para clonar el repositorio

Configuración del backend

1. Clonar el repositorio:

```
git clone [URL_DEL_REPOSITORIO]
cd Back/SpringBoot-TFG
```

2. Crear el archivo `.env` en el directorio raíz del backend con las siguientes variables:

```
DB_URL=jdbc:postgresql://[HOST]:[PORT]/[DATABASE]
DB_USERNAME=[usuario_bd]
DB_PASSWORD=[contraseña_bd]
FIREBASE_CREDENTIALS_BASE64=[credenciales_firebase_en_base64]
```

3. Construir el proyecto:

```
mvn clean package -DskipTests
```

4. Ejecutar el backend:

```
java -jar target/SpringBoot-TFG-*.jar
```

El backend estará disponible en <http://localhost:8080>

5. Acceder a la documentación de la API (Swagger UI):

```
http://localhost:8080/swagger-ui.html
```

Configuración del frontend

1. Acceder al directorio del frontend:

```
cd front/v3/miweb
```

2. Instalar dependencias:

```
npm install
```

3. Configurar las variables de entorno en `src/environments/environment.ts`:

```
export const environment = {  
  production: false,  
  apiUrl: 'http://localhost:8080',  
  firebase: {  
    apiKey: "[FIREBASE_API_KEY]",  
    authDomain: "[FIREBASE_AUTH_DOMAIN]",  
    projectId: "[FIREBASE_PROJECT_ID]",  
    messagingSenderId: "[SENDER_ID]",  
    appId: "[APP_ID]"  
  }  
};
```

4. Iniciar el servidor de desarrollo:

```
ionic serve
```

La aplicación estará disponible en <http://localhost:4200>

Despliegue con Docker Compose

1. Asegurarse de que Docker Desktop está ejecutándose.

2. Desde la raíz del proyecto, ejecutar:

```
docker-compose up --build
```

3. Los servicios quedarán disponibles en:

- Frontend: `http://localhost:80`
- Backend: `http://localhost:8080`
- API Docs: `http://localhost:8080/swagger-ui.html`

11.2 Procedimientos para el control de versiones

El proyecto utiliza **Git** como sistema de control de versiones y **GitHub** como plataforma de alojamiento del repositorio.

Estrategia de ramas (Git Flow simplificado)

- `main`: rama principal con el código estable de producción
- `develop`: rama de integración donde se fusionan las funcionalidades
- `feature/[nombre]`: ramas para el desarrollo de nuevas funcionalidades
- `hotfix/[nombre]`: ramas para corrección urgente de errores en producción

Convención de commits

Se sigue el formato **Conventional Commits**:

```
tipo(ámbito): descripción breve
```

Ejemplos:

```
feat(tickets): añadir filtro de prioridad en lista de tickets
```

```
fix(auth): corregir validación del token expirado
```

```
docs: actualizar README con instrucciones de despliegue
```

```
refactor(diario): extraer lógica de visibilidad a servicio
```

Flujo de trabajo

1. Crear rama `feature/nueva-funcionalidad` desde `develop`
2. Implementar la funcionalidad con commits frecuentes

3. Crear Pull Request hacia `develop`
4. Revisar y fusionar
5. Al finalizar una fase estable, fusionar `develop` en `main`

11.3 Procedimientos para registrar las incidencias

Durante el desarrollo del proyecto, las incidencias (errores, comportamientos inesperados, mejoras pendientes) se registran mediante los siguientes mecanismos:

1. **Issues de GitHub:** Cada incidencia se crea como un issue en el repositorio, incluyendo:
 - Título descriptivo del problema
 - Descripción detallada con pasos para reproducirlo
 - Comportamiento esperado vs comportamiento obtenido
 - Capturas de pantalla o logs si aplica
 - Etiquetas: `bug`, `enhancement`, `documentation`, `question`
 - Asignación y estado: `open`, `in progress`, `closed`
 2. **Sistema de tickets de la propia aplicación:** En el entorno de pruebas, los errores de usuario se registran como tickets de soporte dentro de la plataforma, lo que sirve también para validar el propio módulo de tickets.
 3. **Log del sistema:** El backend registra errores en el log de Spring Boot, que puede consultarse en tiempo real durante el desarrollo.
-

12. DOCUMENTACIÓN DE EJECUCIÓN Y PLAN DE CALIDAD

12.1 Procedimientos operativos

Inicio del sistema en producción

El sistema en producción opera mediante Docker Compose, lo que garantiza el arranque ordenado de los contenedores:

1. La base de datos PostgreSQL arranca primero (con health check configurado).

2. El backend Spring Boot arranca una vez que la base de datos está disponible.
3. El frontend Nginx arranca independientemente y sirve la SPA compilada.

Comprobación del estado de los contenedores:

```
docker-compose ps
docker-compose logs backend
docker-compose logs frontend
```

Reinicio de un servicio específico:

```
docker-compose restart backend
```

Actualización del sistema:

```
git pull origin main
docker-compose up --build -d
```

Copias de seguridad de la base de datos

Se recomienda realizar copias de seguridad periódicas de la base de datos:

```
# Backup de PostgreSQL
pg_dump -h [HOST] -U [USER] -d [DATABASE] > backup_$(date +%Y%m%d).sql

# Restauración
psql -h [HOST] -U [USER] -d [DATABASE] < backup_YYYYMMDD.sql
```

12.2 Registro de pruebas

Se han realizado pruebas en las siguientes categorías:

Pruebas funcionales del backend (mediante Swagger UI)

ID	Endpoint	Método	Descripción	Resultado
PF-01	/api/auth/google	POST	Autenticación con token Firebase válido	OK
PF-02	/api/auth/google	POST	Autenticación con token inválido → 401	OK
PF-03	/api/usuarios/perfil	GET	Obtener perfil autenticado	OK

ID	Endpoint	Método	Descripción	Resultado
PF-04	/api/usuarios/perfil	PUT	Actualizar datos del perfil	OK
PF-05	/api/diarios	POST	Crear diario con visibilidad privada	OK
PF-06	/api/diarios/	GET	Obtener diario propio	OK
PF-07	/api/diarios/	GET	Intentar acceder a diario ajeno privado → 403	OK
PF-08	/api/tickets	POST	Crear ticket con datos válidos	OK
PF-09	/api/tickets	POST	Crear ticket con campos vacíos → 400	OK
PF-10	/api/tickets//estado	PUT	Cambiar estado (ADMIN)	OK
PF-11	/api/tickets//estado	PUT	Cambiar estado (USER) → 403	OK
PF-12	/api/conversaciones	POST	Crear conversación individual	OK
PF-13	/api/mensajes	POST	Enviar mensaje en conversación propia	OK
PF-14	/api/eventos	POST	Crear evento con fecha válida	OK
PF-15	/api/auditoria	GET	Listar auditoría (ADMIN)	OK
PF-16	/api/auditoria	GET	Listar auditoría (USER) → 403	OK

Pruebas de integración frontend–backend

ID	Escenario	Resultado
PI-01	Login con Google → redirección al dashboard	OK
PI-02	Crear ticket desde frontend → aparece en lista	OK
PI-03	Enviar mensaje → aparece en conversación del receptor	OK
PI-04	Cambiar estado de ticket (admin) → notificación al creador	OK
PI-05	Token expirado → interceptor renueva automáticamente	OK
PI-06	Usuario sin rol ADMIN accede a /admin-profile → redirección	OK

Pruebas de validación de datos

ID	Campo	Caso de prueba	Resultado
PV-01	Título del ticket	Vacío → error 400	OK
PV-02	Contenido del diario	Vacío → error 400	OK
PV-03	Email del usuario	Formato inválido → rechazado	OK

ID	Campo	Caso de prueba	Resultado
PV-04	Fecha del evento	Fecha pasada → aceptada (sin restricción de negocio)	OK

12.3 Indicadores de calidad

Se han definido los siguientes indicadores de calidad (KPI) para el proyecto:

Indicador	Objetivo	Resultado obtenido
Cobertura de requisitos funcionales implementados	100% de los RF definidos	100% (14 módulos funcionales)
Endpoints de API documentados en Swagger	>90%	100%
Tiempo medio de respuesta de la API (entorno local)	< 500ms	~150ms promedio
Pruebas funcionales pasadas	>95%	100% de las definidas
Validación de datos en entrada	100% de los campos obligatorios	100%
Código estructurado en capas	Arquitectura por capas completa	Cumplido
Despliegue containerizado funcional	Docker Compose ejecutable	Cumplido

12.4 Métodos de verificación

Los métodos utilizados para verificar la calidad del sistema han sido:

- Revisión de código:** Comprobación manual de la correcta implementación de la arquitectura por capas, nomenclatura de endpoints REST y validaciones.
- Pruebas con Swagger UI:** Todos los endpoints han sido probados manualmente a través de la interfaz Swagger, verificando los códigos de respuesta HTTP y la estructura de los datos devueltos.
- Pruebas end-to-end manuales:** Se han simulado flujos de usuario completos (registro → creación de tickets → cambio de estado → notificación) para verificar la correcta integración.
- Revisión de seguridad:** Se ha comprobado que los endpoints protegidos devuelven 401 sin token, 403 con token insuficiente y que los recursos de un usuario no son accesibles por otro.
- Pruebas de despliegue:** Se ha verificado que el sistema arranca correctamente mediante `docker-compose up` y que todos los servicios son accesibles.

13. DISTRIBUCIÓN

13.1 Tecnología de distribución

La distribución de WorkSpace se realiza mediante **contenedores Docker**, orquestados con **Docker Compose**. Este enfoque garantiza que el sistema puede ejecutarse de forma idéntica en cualquier entorno que tenga Docker instalado, eliminando los problemas de "funciona en mi máquina".

Los componentes del sistema se distribuyen como:

- **Imagen Docker del backend:** Construida sobre `eclipse-temurin:17-jre-alpine` (imagen Java mínima). El proceso de build usa Maven para compilar el JAR y Docker para empaquetarlo.
- **Imagen Docker del frontend:** Construida en dos etapas. La primera usa `node:20-alpine` para compilar la aplicación Angular/Ionic. La segunda usa `nginx:alpine` para servir los archivos estáticos compilados.
- **Imagen de PostgreSQL:** Se usa la imagen oficial `postgres:16-alpine` desde Docker Hub.

13.2 Descripción del proceso de distribución

Proceso de build y distribución

Paso 1 — Build del backend:

```
# Dockerfile del backend (multi-stage)
FROM maven:3.9-eclipse-temurin-17 AS builder
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Paso 2 — Build del frontend:

```
# Dockerfile del frontend (multi-stage)
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build -- --configuration production

FROM nginx:alpine
COPY --from=builder /app/www /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

Paso 3 — Docker Compose:

```
version: '3.8'
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: workspacedb
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER}"]
      interval: 10s
      timeout: 5s
      retries: 5

  backend:
    build: ./Back/SpringBoot-TFG
    ports:
      - "8080:8080"
    environment:
      DB_URL: jdbc:postgresql://postgres:5432/workspacedb
      DB_USERNAME: ${DB_USER}
      DB_PASSWORD: ${DB_PASSWORD}
      FIREBASE_CREDENTIALS_BASE64: ${FIREBASE_CREDENTIALS_BASE64}
    depends_on:
      postgres:
        condition: service_healthy

  frontend:
    build: ./front/v3/miweb
    ports:
      - "80:80"
    depends_on:
      - backend

volumes:
  postgres_data:
```

Paso 4 — Despliegue:


```
# Construir y arrancar todos los servicios
docker-compose up --build -d

# Verificar estado
docker-compose ps

# Ver logs en tiempo real
docker-compose logs -f
```

14. MANUALES

14.1 Manual de instalación

Instalación para entorno de producción (Docker)

Prerrequisitos:

- Servidor Linux (Ubuntu 22.04 LTS recomendado) o Windows con Docker Desktop
- Docker Engine 24.x o superior
- Docker Compose 2.x o superior
- Acceso a Internet para descargar imágenes base

Pasos de instalación:

1. Clonar el repositorio en el servidor:

```
git clone [URL_REPOSITORIO] workspace
cd workspace
```

2. Crear el archivo de variables de entorno:

```
cp .env.example .env
nano .env
```

Configurar las variables:

```
DB_USER=workspace_user
DB_PASSWORD=contraseña_segura
FIREBASE_CREDENTIALS_BASE64=base64_de_las_credenciales_firebase
```

3. Construir y arrancar los contenedores:

```
docker-compose up --build -d
```

4. Verificar que el sistema está operativo:

```
docker-compose ps
# Debe mostrar los tres servicios con estado "running"
```

5. Acceder a la aplicación:

- Frontend: `http://[IP_DEL_SERVIDOR]`
- API Backend: `http://[IP_DEL_SERVIDOR]:8080`
- Swagger UI: `http://[IP_DEL_SERVIDOR]:8080/swagger-ui.html`

Detener el sistema:

```
docker-compose down
```

Actualizar a una nueva versión:

```
git pull origin main
docker-compose up --build -d
```

14.2 Manual de usuario

Registro e inicio de sesión

1. Acceder a la aplicación desde el navegador.
2. En la página de inicio, hacer clic en el botón **"Iniciar sesión con Google"**.
3. Seleccionar la cuenta de Google deseada en el popup que se abrirá.
4. Una vez autenticado, el sistema redirigirá automáticamente al dashboard principal.

Nota: El primer acceso creará automáticamente una cuenta de usuario con rol básico.

Gestión del perfil

1. Acceder a **Perfil** desde el menú lateral izquierdo.
2. Editar los campos de nombre, biografía y foto de perfil.
3. Configurar las preferencias de contacto (permitir/no permitir que otros usuarios inicien conversaciones).
4. Guardar los cambios pulsando el botón **"Guardar"**.

Uso del módulo de diarios

Crear un nuevo diario:

1. Acceder a **Diario** desde el menú lateral.
2. Seleccionar o crear un tema desde la sección de temas.
3. Pulsar "**Nueva entrada**".
4. Escribir el contenido (soporta formato Markdown).
5. Seleccionar la visibilidad: Privada / Pendiente de revisión / Pública.
6. Pulsar "**Guardar**".

Colaborar en un tema:

1. En la sección de temas, pulsar el botón de invitar colaboradores.
2. Buscar al usuario por nombre o email.
3. Enviar la invitación. El usuario recibirá una notificación y podrá aceptar o rechazar.

Uso del módulo de tickets

Crear un ticket de soporte:

1. Acceder a **Tickets** desde el menú lateral.
2. Pulsar "**Nuevo ticket**".
3. Completar el título, descripción y seleccionar la prioridad (Alta/Media/Baja).
4. Pulsar "**Enviar**".

Seguimiento de tickets:

- La lista de tickets muestra el estado actual de cada solicitud.
- Al hacer clic en un ticket se pueden ver los comentarios y el historial de cambios.
- Los usuarios pueden añadir comentarios para proporcionar información adicional al staff.

Uso de la mensajería

Iniciar una conversación:

1. Acceder a **Mensajes** desde el menú lateral.
2. Pulsar "**Nueva conversación**".
3. Buscar y seleccionar el usuario o usuarios con los que conversar.
4. Asignar un título (opcional para conversaciones grupales).
5. Empezar a escribir mensajes.

Uso del calendario de eventos

1. Acceder a **Eventos** desde el menú lateral.
2. El calendario muestra todos los eventos (propios y públicos).
3. Para crear un evento: pulsar "**Nuevo evento**", completar los datos y guardar.
4. Los eventos privados sólo son visibles para el creador; los públicos son visibles para todos.

Panel de administración (solo ADMIN y STAFF)

El panel de administración está accesible mediante la sección `/admin-profile` y contiene:

- **Usuarios:** Lista completa de usuarios con opciones de editar rol/departamento y eliminar.
 - **Tickets (admin):** Gestión global de todos los tickets; cambiar estado y prioridad.
 - **Diarios (admin):** Revisar y aprobar diarios pendientes de publicación.
 - **Mensajes soporte:** Bandeja de entrada del servicio de soporte al usuario.
 - **Auditoría:** Registro completo de acciones del sistema con filtros por tipo y severidad.
 - **Personal:** Gestión de departamentos y asignación de personal.
-

15. CONCLUSIONES

15.1 Informe final

El proyecto Workspace ha sido completado satisfactoriamente, alcanzando todos los objetivos definidos en la fase de planificación y superando en varios aspectos las expectativas iniciales. Se ha desarrollado una plataforma web full-stack completamente funcional que integra cinco módulos principales (diarios, tickets, mensajería, eventos, auditoría) con un sistema de roles y una interfaz moderna y responsiva.

La aplicación ha sido desplegada en un entorno real mediante Docker Compose, lo que demuestra la capacidad para llevar un proyecto desde el análisis hasta la producción, cubriendo todas las fases del ciclo de vida del software.

15.2 Resultados obtenidos

Los principales resultados del proyecto son:

- **Backend:** 15 controladores REST, 13 servicios, 16 entidades JPA, sistema de seguridad con Firebase JWT, auditoría con Spring AOP y documentación completa con Swagger/OpenAPI.
- **Frontend:** Aplicación Angular/Ionic responsiva con 20+ páginas, sistema de routing con guards de autenticación y rol, interceptor HTTP para tokens, integración con Firebase Authentication y FCM.
- **Base de datos:** Esquema PostgreSQL normalizado con 16 tablas, índices en campos críticos e integridad referencial completa.
- **Despliegue:** Sistema containerizado con Docker Compose listo para producción, con Nginx sirviendo el frontend y Spring Boot el backend.
- **Calidad:** 100% de los requisitos funcionales implementados, 100% de las pruebas funcionales definidas pasadas.

15.3 Viabilidad del proyecto

El proyecto es técnicamente viable y económicamente accesible:

- **Técnicamente:** La arquitectura elegida (Spring Boot + Angular + PostgreSQL + Docker) es una combinación madura y ampliamente utilizada en la industria. Las tecnologías son estables, tienen soporte activo y una comunidad grande.
- **Económicamente:** El coste de los servicios utilizados en la versión gratuita (Firebase Spark, Neon free tier) es cero. Para un entorno de producción real, el coste mensual estimado sería:
 - VPS básico (2 vCores, 4 GB RAM): ~10–15 €/mes
 - PostgreSQL gestionado (Neon Basic): ~20 €/mes
 - Firebase: gratuito para el volumen esperado
 - Total estimado: ~30–35 €/mes
- **Operativamente:** El sistema requiere mantenimiento mínimo: actualizaciones de dependencias trimestrales y backups periódicos de la base de datos.

15.4 Mejoras futuras

Se han identificado las siguientes líneas de mejora para versiones futuras:

1. **Mensajería en tiempo real:** Implementar WebSockets (Spring WebSocket + STOMP) para actualización instantánea de mensajes sin necesidad de recarga.
2. **Sistema de archivos adjuntos:** Integrar almacenamiento de archivos (Firebase Storage o S3-compatible) para permitir adjuntar documentos a diarios y tickets.
3. **Notificaciones por email:** Integrar un servicio de envío de emails (SendGrid, AWS SES) para notificaciones adicionales.
4. **Tests automatizados:** Implementar tests unitarios con JUnit 5/Mockito para el backend y tests E2E con Cypress para el frontend, con cobertura >80%.
5. **Pipeline CI/CD:** Configurar GitHub Actions para automatizar el build, tests y despliegue en cada push a la rama principal.
6. **Internacionalización (i18n):** Añadir soporte multi-idioma mediante Angular i18n.
7. **Analítica y reportes:** Panel de estadísticas para administradores: tickets por período, usuarios activos, contenido más popular.
8. **Escalabilidad horizontal:** Migrar a una arquitectura de microservicios y despliegue en Kubernetes para soportar mayor carga.

9. **Aplicación nativa:** Compilar el frontend Ionic como aplicación nativa para Android e iOS mediante Capacitor y publicar en tiendas.

10. **Búsqueda full-text:** Integrar Elasticsearch para búsqueda avanzada en diarios, tickets y mensajes.

16. ANEXOS

Anexo A: Diagrama Entidad–Relación completo

Tabla: usuario

Campo	Tipo	Restricciones
id	BIGINT	PK, AUTO_INCREMENT
firebase_uid	VARCHAR(255)	UNIQUE, NOT NULL, INDEX
email	VARCHAR(255)	UNIQUE, NOT NULL, INDEX
nombre	VARCHAR(255)	NOT NULL
biografia	TEXT	NULLABLE
foto_perfil	VARCHAR(500)	NULLABLE
rol_id	BIGINT	FK → rol(id), NOT NULL
departamento_id	BIGINT	FK → departamento(id), NULLABLE
fcm_token	VARCHAR(500)	NULLABLE
permite_contacto	BOOLEAN	DEFAULT TRUE
motivo_no_contacto	TEXT	NULLABLE
fecha_creacion	TIMESTAMP	NOT NULL

Tabla: rol

Campo	Tipo	Restricciones
id	BIGINT	PK, AUTO_INCREMENT
nombre	VARCHAR(50)	UNIQUE, NOT NULL (USER/STAFF/ADMIN)

Tabla: departamento

Campo	Tipo	Restricciones
id	BIGINT	PK, AUTO_INCREMENT
nombre	VARCHAR(255)	NOT NULL

Tabla: diario

Campo	Tipo	Restricciones
-------	------	---------------

Campo	Tipo	Restricciones
id	BIGINT	PK, AUTO_INCREMENT
contenido	TEXT	NOT NULL
fecha_creacion	TIMESTAMP	NOT NULL
visibilidad	VARCHAR(20)	NOT NULL (PRIVADO/PENDIENTE/PUBLICO)
usuario_id	BIGINT	FK → usuario(id), NOT NULL, INDEX
tema_id	BIGINT	FK → diario_tema(id), NULLABLE
revisado_por	BIGINT	FK → usuario(id), NULLABLE
revision_comentario	TEXT	NULLABLE
revision_en	TIMESTAMP	NULLABLE

Tabla: diario_tema

Campo	Tipo	Restricciones
id	BIGINT	PK, AUTO_INCREMENT
titulo	VARCHAR(255)	NOT NULL
descripcion	TEXT	NULLABLE
usuario_id	BIGINT	FK → usuario(id), NOT NULL, INDEX

Tabla: diario_colaboracion

Campo	Tipo	Restricciones
id	BIGINT	PK, AUTO_INCREMENT
tema_id	BIGINT	FK → diario_tema(id), NOT NULL
usuario_id	BIGINT	FK → usuario(id), NOT NULL
estado	VARCHAR(20)	NOT NULL (PENDIENTE/ACEPTADA/RECHAZADA)
fecha_invitacion	TIMESTAMP	NOT NULL

Tabla: diario_comentario

Campo	Tipo	Restricciones
id	BIGINT	PK, AUTO_INCREMENT
diario_id	BIGINT	FK → diario(id), NOT NULL
autor_id	BIGINT	FK → usuario(id), NOT NULL
texto	TEXT	NOT NULL
fecha	TIMESTAMP	NOT NULL

Tabla: ticket

Campo	Tipo	Restricciones
id	BIGINT	PK, AUTO_INCREMENT
titulo	VARCHAR(255)	NOT NULL
descripcion	TEXT	NOT NULL
estado	VARCHAR(20)	NOT NULL (abierto/en_progreso/resuelto)
prioridad	VARCHAR(10)	NOT NULL (ALTA/MEDIA/BAJA)

Campo	Tipo	Restricciones
usuario_id	BIGINT	FK → usuario(id), NOT NULL
fecha_creacion	TIMESTAMP	NOT NULL

Tabla: ticket_comentario

Campo	Tipo	Restricciones
id	BIGINT	PK, AUTO_INCREMENT
ticket_id	BIGINT	FK → ticket(id), NOT NULL
usuario_id	BIGINT	FK → usuario(id), NOT NULL
texto	TEXT	NOT NULL
fecha_envio	TIMESTAMP	NOT NULL

Tabla: ticket_historico

Campo	Tipo	Restricciones
id	BIGINT	PK, AUTO_INCREMENT
ticket_id	BIGINT	FK → ticket(id), NOT NULL
estado_anterior	VARCHAR(20)	NOT NULL
estado_nuevo	VARCHAR(20)	NOT NULL
usuario_id	BIGINT	FK → usuario(id), NOT NULL
comentario	TEXT	NULLABLE
fecha	TIMESTAMP	NOT NULL

Tabla: conversacion

Campo	Tipo	Restricciones
id	BIGINT	PK, AUTO_INCREMENT
titulo	VARCHAR(255)	NULLABLE
tipo	VARCHAR(20)	NOT NULL (individual/grupal)
creado_por	BIGINT	FK → usuario(id), NOT NULL, INDEX
fecha_creacion	TIMESTAMP	NOT NULL

Tabla: conversacion_participante

Campo	Tipo	Restricciones
conversacion_id	BIGINT	FK → conversacion(id), PK compuesta
usuario_id	BIGINT	FK → usuario(id), PK compuesta
fecha_entrada	TIMESTAMP	NOT NULL
fecha_salida	TIMESTAMP	NULLABLE

Tabla: mensaje

Campo	Tipo	Restricciones
id	BIGINT	PK, AUTO_INCREMENT

Campo	Tipo	Restricciones
conversacion_id	BIGINT	FK → conversacion(id), NOT NULL, INDEX
autor_id	BIGINT	FK → usuario(id), NOT NULL, INDEX
texto	TEXT	NOT NULL
fecha_envio	TIMESTAMP	NOT NULL, INDEX
leido_en	TIMESTAMP	NULLABLE

Tabla: evento

Campo	Tipo	Restricciones
id	BIGINT	PK, AUTO_INCREMENT
titulo	VARCHAR(255)	NOT NULL
descripcion	TEXT	NULLABLE
fecha_evento	DATE	NOT NULL
hora_evento	TIME	NULLABLE
visibilidad	VARCHAR(20)	NOT NULL (PRIVADO/PUBLICO)
usuario_id	BIGINT	FK → usuario(id), NOT NULL

Tabla: notificacion

Campo	Tipo	Restricciones
id	BIGINT	PK, AUTO_INCREMENT
usuario_id	BIGINT	FK → usuario(id), NOT NULL
mensaje	TEXT	NOT NULL
fecha	TIMESTAMP	NOT NULL
leida	BOOLEAN	DEFAULT FALSE

Tabla: auditoria

Campo	Tipo	Restricciones
id	BIGINT	PK, AUTO_INCREMENT
accion	VARCHAR(50)	NOT NULL (LOGIN/CREAR/ELIMINAR/etc.)
recurso	VARCHAR(50)	NOT NULL (Usuario/Ticket/Evento/etc.)
descripcion	TEXT	NULLABLE
severidad	VARCHAR(20)	NOT NULL (INFO/WARNING/DANGER)
fecha	TIMESTAMP	NOT NULL
usuario_id	BIGINT	FK → usuario(id), NULLABLE
usuario_email	VARCHAR(255)	NULLABLE

Anexo B: Descripción detallada de la API REST

Autenticación

Método	Endpoint	Auth	Descripción
POST	/api/auth/google	NO	Login con token Firebase

Usuarios

Método	Endpoint	Auth	Roles	Descripción
GET	/api/usuarios	JWT	ADMIN	Listar todos los usuarios
GET	/api/usuarios/	JWT	ADMIN	Obtener usuario por ID
GET	/api/usuarios/perfil	JWT	ANY	Obtener perfil propio
GET	/api/usuarios/departamentos	JWT	ANY	Listar departamentos
GET	/api/usuarios/buscar	JWT	ANY	Buscar usuarios
POST	/api/usuarios/registro	JWT	ANY	Registrar usuario
PUT	/api/usuarios/perfil	JWT	ANY	Actualizar perfil propio
PUT	/api/usuarios/	JWT	ADMIN	Actualizar usuario
DELETE	/api/usuarios/perfil	JWT	ANY	Eliminar cuenta propia
DELETE	/api/usuarios/	JWT	ADMIN	Eliminar usuario

Diarios

Método	Endpoint	Auth	Roles	Descripción
GET	/api/diarios	JWT	ANY	Listar diarios visibles
GET	/api/diarios/mis-diarios	JWT	ANY	Mis diarios
GET	/api/diarios/publicos	JWT	ANY	Diarios públicos
GET	/api/diarios/	JWT	ANY	Ver diario (con control de acceso)
GET	/api/diarios//comentarios	JWT	ANY	Comentarios del diario
GET	/api/diarios/usuario/	JWT	STAFF,ADMIN	Diarios de un usuario
POST	/api/diarios	JWT	ANY	Crear diario
POST	/api/diarios//comentarios	JWT	ANY	Añadir comentario
PUT	/api/diarios/	JWT	ANY	Actualizar diario propio
DELETE	/api/diarios/	JWT	ANY	Eliminar diario propio

Tickets

Método	Endpoint	Auth	Roles	Descripción
GET	/api/tickets	JWT	STAFF,ADMIN	Listar todos los tickets
GET	/api/tickets/mis-tickets	JWT	ANY	Mis tickets
POST	/api/tickets	JWT	ANY	Crear ticket
PATCH	/api/tickets/	JWT	ADMIN	Actualizar ticket (admin)
PUT	/api/tickets//estado	JWT	STAFF,ADMIN	Cambiar estado del ticket

Mensajería

Método	Endpoint	Auth	Roles	Descripción
--------	----------	------	-------	-------------

Método	Endpoint	Auth	Roles	Descripción
GET	/api/conversaciones	JWT	ANY	Mis conversaciones
POST	/api/conversaciones	JWT	ANY	Crear conversación
DELETE	/api/conversaciones/	JWT	ANY	Eliminar conversación
GET	/api/mensajes/conversacion/	JWT	ANY	Mensajes de conversación
POST	/api/mensajes	JWT	ANY	Enviar mensaje
PUT	/api/mensajes/leer-todo/	JWT	ANY	Marcar como leídos

Eventos

Método	Endpoint	Auth	Roles	Descripción
GET	/api/eventos	JWT	ANY	Listar eventos
POST	/api/eventos	JWT	ANY	Crear evento
DELETE	/api/eventos/	JWT	ANY	Eliminar evento

Notificaciones y Auditoría

Método	Endpoint	Auth	Roles	Descripción
GET	/api/notificaciones	JWT	ANY	Mis notificaciones
DELETE	/api/notificaciones/	JWT	ANY	Eliminar notificación
GET	/api/auditoria	JWT	ADMIN	Listar auditoría
GET	/api/auditoria/	JWT	ADMIN	Ver registro de auditoría

Anexo C: Casos de prueba

CP-001: Autenticación correcta con Google

- **Precondición:** Cuenta de Google válida disponible
- **Entrada:** Token JWT válido de Firebase
- **Resultado esperado:** HTTP 200 + datos del usuario (o creación si es nuevo)
- **Resultado obtenido:** HTTP 200 ☒

CP-002: Acceso a endpoint protegido sin token

- **Precondición:** Ninguna
- **Entrada:** GET /api/usuarios/perfil sin cabecera Authorization
- **Resultado esperado:** HTTP 401 Unauthorized
- **Resultado obtenido:** HTTP 401 ☒

CP-003: Acceso a funcionalidad de admin con rol USER

- **Precondición:** Usuario autenticado con rol USER
- **Entrada:** GET /api/auditoria con token JWT de usuario USER
- **Resultado esperado:** HTTP 403 Forbidden

- **Resultado obtenido:** HTTP 403 ☐

CP-004: Crear ticket con campos válidos

- **Precondición:** Usuario autenticado
- **Entrada:** POST /api/tickets con {"titulo": "Error en login", "descripcion": "No puedo acceder", "prioridad": "ALTA"}
- **Resultado esperado:** HTTP 201 Created + ticket creado
- **Resultado obtenido:** HTTP 201 ☐

CP-005: Crear ticket con campos vacíos

- **Precondición:** Usuario autenticado
- **Entrada:** POST /api/tickets con {"titulo": "", "descripcion": ""}
- **Resultado esperado:** HTTP 400 Bad Request + mensaje de error
- **Resultado obtenido:** HTTP 400 ☐

CP-006: Acceder a diario privado ajeno

- **Precondición:** Usuario A autenticado; diario privado creado por Usuario B
- **Entrada:** GET /api/diarios/ con token de A
- **Resultado esperado:** HTTP 403 Forbidden
- **Resultado obtenido:** HTTP 403 ☐

CP-007: Flujo completo de ticket (crear → resolver)

- **Precondición:** Usuario USER y usuario ADMIN autenticados
- **Paso 1:** USER crea ticket → Estado: abierto ☐
- **Paso 2:** ADMIN cambia estado a en_progreso → Historial registrado ☐
- **Paso 3:** ADMIN cambia estado a resuelto → Notificación enviada a USER ☐
- **Resultado:** Ciclo de vida completo del ticket funcionando ☐

17. ÍNDICE DE TABLAS E IMÁGENES

Tablas

- Tabla 1: Análisis comparativo de aplicaciones similares (§3.2)
- Tabla 2: Usuarios destinatarios del sistema (§4.3)
- Tabla 3: Requisitos funcionales — Gestión de usuarios (§7.1)

- Tabla 4: Requisitos funcionales — Sistema de diarios (§7.1)
- Tabla 5: Requisitos funcionales — Tickets (§7.1)
- Tabla 6: Stack tecnológico completo (§10.2)
- Tabla 7: Herramientas de desarrollo (§8.3)
- Tabla 8: Servicios en la nube (§8.3)
- Tabla 9: Identificación y evaluación de riesgos (§9.1)
- Tabla 10: Plan de mitigación de riesgos (§9.3)
- Tabla 11: Pruebas funcionales del backend (§12.2)
- Tabla 12: Pruebas de integración (§12.2)
- Tabla 13: Indicadores de calidad (§12.3)
- Tabla 14: Resumen de endpoints de la API — Autenticación (Anexo B)
- Tabla 15: Resumen de endpoints de la API — Usuarios (Anexo B)
- Tabla 16: Resumen de endpoints de la API — Diarios (Anexo B)
- Tabla 17: Resumen de endpoints de la API — Tickets (Anexo B)
- Tabla 18: Resumen de endpoints de la API — Mensajería (Anexo B)
- Tabla 19–22: Tablas del modelo de datos — [todas las entidades] (Anexo A)

Imágenes

- Figura 1: Diagrama de arquitectura global del sistema (§10.2)
- Figura 2: Diagrama de casos de uso general (§10.3)
- Figura 3: Diagrama de flujo de autenticación (§10.3)
- Figura 4: Diagrama de flujo del ciclo de vida de un ticket (§10.3)
- Figura 5: Modelo Entidad–Relación simplificado (§10.3)
- Figura 6: Diagrama de Gantt del proyecto (§8.2)
- Figura 7: Estructura de capas del backend (§10.2)
- Figura 8–28: Capturas de pantalla de la aplicación [adjuntas en /documentación]

18. BIBLIOGRAFÍA Y REFERENCIAS

Documentación oficial

- **Spring Boot Documentation** (3.5.x). Pivotal/VMware. <https://docs.spring.io/spring-boot/docs/current/reference/html/> (<https://docs.spring.io/spring-boot/docs/current/reference/html/>).
- **Spring Security Reference** (6.x). Spring Framework. <https://docs.spring.io/spring-security/reference/> (<https://docs.spring.io/spring-security/reference/>).

- **Kotlin Documentation.** JetBrains. <https://kotlinlang.org/docs/home.html> (<https://kotlinlang.org/docs/home.html>)
- **Angular Documentation** (v20). Google. <https://angular.dev/> (<https://angular.dev/>)
- **Ionic Framework Documentation** (v8). Ionic Team. <https://ionicframework.com/docs> (<https://ionicframework.com/docs>)
- **Firestore Documentation** — Authentication & Cloud Messaging. Google. <https://firebase.google.com/docs> (<https://firebase.google.com/docs>)
- **PostgreSQL Documentation** (16). The PostgreSQL Global Development Group. <https://www.postgresql.org/docs/16/> (<https://www.postgresql.org/docs/16/>)
- **Docker Documentation.** Docker Inc. <https://docs.docker.com/> (<https://docs.docker.com/>)
- **Docker Compose Reference.** Docker Inc. <https://docs.docker.com/compose/> (<https://docs.docker.com/compose/>)
- **SpringDoc OpenAPI Documentation.** <https://springdoc.org/> (<https://springdoc.org/>)
- **Syncfusion EJ2 Angular Schedule** (Documentation). Syncfusion. <https://ej2.syncfusion.com/angular/documentation/schedule/> (<https://ej2.syncfusion.com/angular/documentation/schedule/>)

Libros y recursos técnicos

- Craig Walls. *Spring in Action* (6ª edición). Manning Publications, 2022.
- Venkat Subramaniam. *Programming Kotlin*. Pragmatic Bookshelf, 2019.
- Sam Newman. *Building Microservices* (2ª edición). O'Reilly Media, 2021.
- Roy T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, UC Irvine, 2000. (Trabajo original que define REST)

Artículos y recursos en línea

- Baeldung. *Spring Security with JWT*. <https://www.baeldung.com/spring-security-oauth-jwt> (<https://www.baeldung.com/spring-security-oauth-jwt>)
- Auth0. *Introduction to JSON Web Tokens*. <https://jwt.io/introduction> (<https://jwt.io/introduction>)
- Martin Fowler. *Patterns of Enterprise Application Architecture*. <https://martinfowler.com/eaCatalog/> (<https://martinfowler.com/eaCatalog/>)
- Stack Overflow. *Comunidad de desarrollo de software*. <https://stackoverflow.com/> (<https://stackoverflow.com/>)

- GitHub. *Repositorio de código del proyecto*. [URL_DEL_REPOSITORIO]

Normativa y legislación

- **Reglamento (UE) 2016/679** del Parlamento Europeo y del Consejo, de 27 de abril de 2016 (Reglamento General de Protección de Datos — RGPD). Diario Oficial de la Unión Europea.
- **Ley Orgánica 3/2018, de 5 de diciembre**, de Protección de Datos Personales y garantía de los derechos digitales (LOPDGDD). BOE-A-2018-16673.
- **W3C Web Content Accessibility Guidelines (WCAG) 2.1**. World Wide Web Consortium, 2018.
<https://www.w3.org/TR/WCAG21/> (<https://www.w3.org/TR/WCAG21/>).

Documento generado para el Trabajo de Fin de Grado — Ciclo Superior DAM — Curso 2024–2025

Autor: [NOMBRE COMPLETO DEL ALUMNO]

Centro: [NOMBRE DEL CENTRO EDUCATIVO]